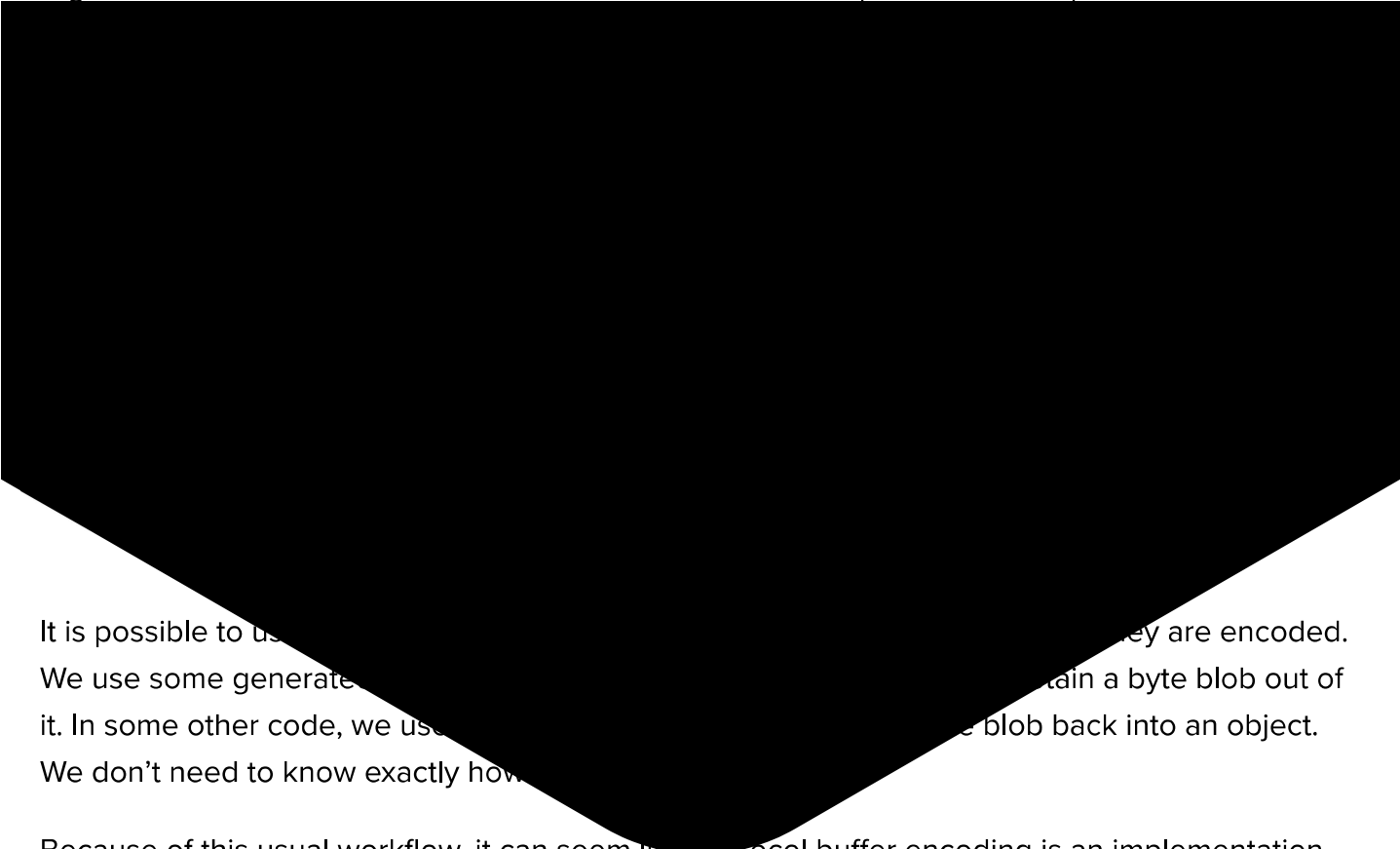




It is possible to use `ByteBuffer` to encode and decode objects. They are encoded. We use some generated code to encode an object and obtain a byte blob out of it. In some other code, we use `ByteBuffer` to decode the blob back into an object. We don't need to know exactly how the encoding works.

Because of this usual workflow, it can seem like protocol buffer encoding is an implementation detail. And when something is an implementation detail, the healthy thing to do is not depend on it, since it could change at any point at the author's discretion.

This is not the truth. Protocol buffer encoding can in fact be seen more like part of protocol buffers API surface than an implementation detail. As such, it is very stable. Knowing how protocol buffer encoding works and how to take advantage of it enables some interesting advanced usages.

We will first describe how some of the encoding and decoding of protocol buffers work. Then we will see some examples of needs that can be covered by taking advantage of that knowledge.

i This document covers partial explanations of how protocol buffer encoding and decoding works. It will be enough for our purposes. For a fuller understanding of protocol buffer encoding read the official Encoding documentation (<https://developers.google.com/protocol-buffers/docs/encoding>) and Language guide (<https://developers.google.com/protocol-buffers/docs/proto3>).

Protocol buffer encoding primer

Let's start with a simplified explanation of how protocol buffer messages are encoded.

Say we have the following message definition:

```
message Thing {  
  fixed32 a = 1;  
  string b = 2;  
}
```

We create a `Thing` in some program, in some language:

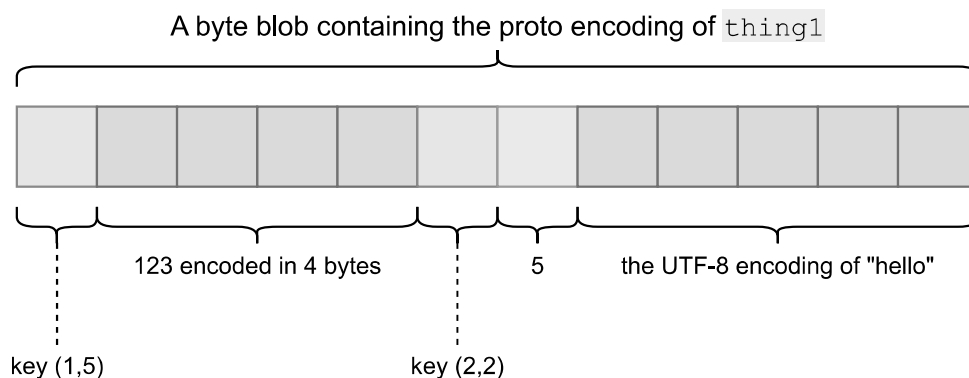
```
var thing1 = Thing(a: 123, b: "hello")
```

We then encode `thing1` with a protocol buffer class. For each field, the protocol buffer will encode a key and a value. The key contains information about the field number, and the field type.

In this case, encoding `thing1` yields:

- The first byte is the key for field `a` : it is composed of the field number, which here is `1` , and a number representing the type `fixed32` , which by convention is `5`
- The next four bytes contain the value of field `a` : the fixed 32 bit representation of `123`
- The next byte is the key for field `b` : it is comprised of number `2` , followed by the number used to represent type `string`, which is also `2`
- The next few bytes contain the value of field `b` . Because `b` contains a string, it is a variable length field. So its value is decomposed in two parts:
 - One byte representing the number `5` , which is the length of the text `hello`
 - Five bytes containing the UTF-8 encoding of `hello`

The resulting byte blob yielded by protocol buffer encoding is the concatenation of all of these bytes:



If we give this byte blob to a protocol buffer decoder, this is how it will proceed:

1. It knows that the first byte should be a key. It reads it and finds field number `1` and field type `5` . The decoder knows our schema, so it knows that field `1` corresponds to field `a` in code. The decoder also knows that type `5` corresponds to a 32 bit number
2. The decoder reads the next 4 bytes and interprets them as a 32 bit number, yielding `123`
3. The next byte should be a key. It contains field number `2` and field type `2` . Thanks to the schema, the decoder knows field number `2` corresponds to `b` .

4. Field type `2` corresponds to a variable length value. So the next byte should contain the length of the value. The next byte is read: it contains the number `5`
5. The decoder now knows the length of the string is `5`. It reads `5` bytes and decodes as UTF-8, yielding the text `hello`

As we can see, fields in a protocol buffer message are encoded and concatenated. There is no special set of bytes used to separate fields. A protocol buffer decoder can separate keys and values as it parses a byte blob because it always knows their lengths. It knows how many bytes a key has. And it knows how many bytes a value has too: a fixed length value uses a fixed number of bytes; a variable length value is prepended by its byte count.

Repeated fields

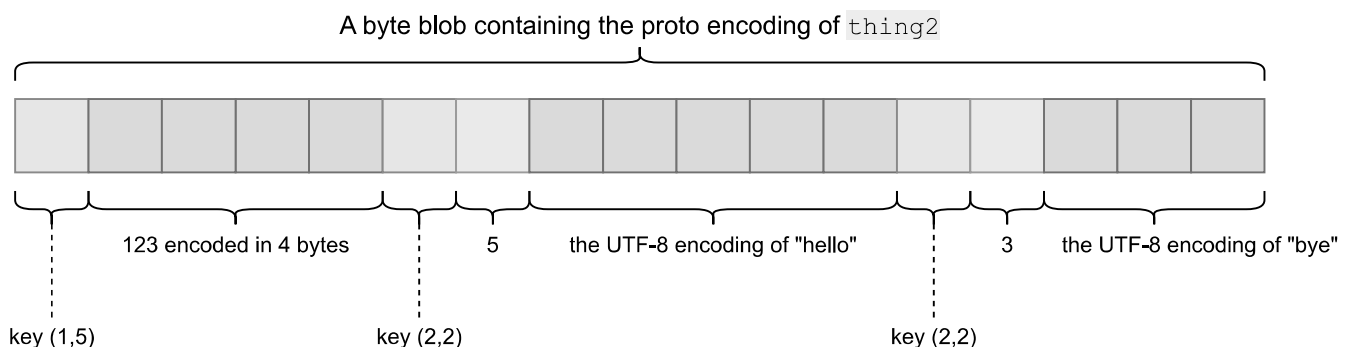
Say we now have the following message type:

```
message Thing {  
  fixed32 a = 1;  
  repeated string b = 2;  
}
```

We make an instance of it:

```
var thing2 = Thing(a: 123, b: ["hello", "bye"])
```

How is this message encoded, now that it has several values in a repeated field? Repeated fields, as their name implies, are encoded by *repeatedly* encoding the same field, with the same key, but different values:



Optional fields

Optional fields are encoded differently than normal fields, in a way that allows knowing whether the field is present or not in a message.

Say we have a schema with a field `fixed32 a = 1;`. We create an object with `a` set to `0`. Zero is the default value for numeric fields. So when we encode our object, field `a` will simply not be written at all in the byte blob, to keep the data as compact as possible.

On the decoding side, the parser will see that `a` is not set, so it will give it the default value of `0`. But the parser has no way to know why is it that `a` was not present in the data:

- It might be because it was explicitly set to `0` so the decoder didn't put it in the data
- Or it might be because it was not set at all, so there was nothing to encode in the data

Sometimes we want to be able to distinguish between these two scenarios, and `optional` exist for that.

We make the field optional: `optional fixed32 a = 1;`. Now, if we encode an object with `a` set to `0`, this time the value `0` will be encoded explicitly in the byte blob. On the decoder side, we can now distinguish two scenarios:

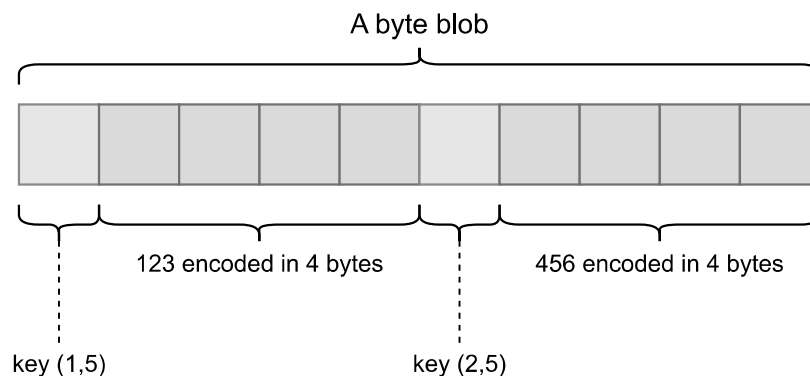
- If `a` is set to `0` in the data, it means it was explicitly set to that value
- If `a` is not present in the data at all, it means it wasn't present in the encoded object

Unknown fields

When a decoder encounters an unknown field in a byte blob, it is expected to ignore it. Say we have this schema:

```
message Thing {  
  fixed32 a = 1;  
}
```

And we receive this blob:



Our decoder will recognize key `1`, corresponding to field `a`, and decode it. But it won't recognize key `2`, which is not in the schema, so it will simply ignore it. Some decoder implementations will put unrecognized fields in a special "unrecognized" section of the decoded object.

The reason for this behavior is backward and forward compatibility. For instance the blob might have been encoded by another program that knows a more recent version of the same schema:

```
message Thing {  
  fixed32 a = 1;  
  fixed32 b = 2;  
}
```

}

Encoding and decoding in protocol buffers have been designed for maximum compatibility:

- An old program should be able to read data made with a new version of the schema without breaking
- An updated program should be able to read data made with an old version of the schema without breaking

There are many changes we can make to a protocol buffer schema in a compatible way. Some examples:

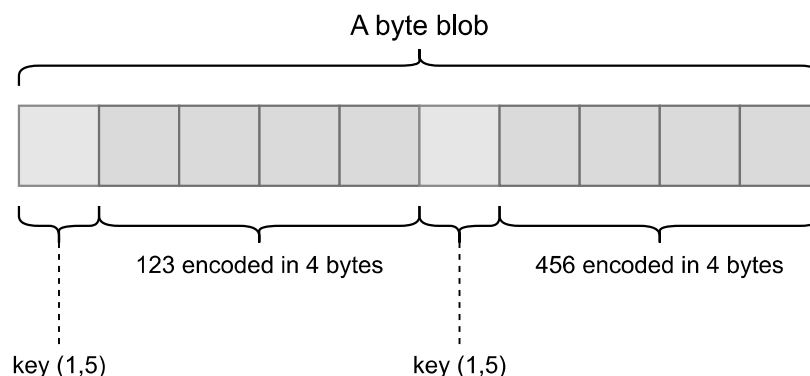
- Add a field: programs with the old schema will ignore the unknown field in new data. Programs with the new schema will use the default value when reading old data that doesn't have the field (or nil if it is an optional field)
- Remove a field: programs with the old schema will use the default value when reading new data that misses the field. Programs with the new schema will ignore the now unknown field in old data
- Rename a field: encoded data uses field numbers, not field names, so we can change a field's name at will as long as we keep the same field number
- Rename a message name: similarly, message names don't appear in encoded data

Duplicate fields

Normally, a protocol buffer encoder only encodes each field of an object once. But decoders are expected to accept a byte blob that contains duplicates of a given field. For instance if we have this schema:

```
message Thing {  
  fixed32 a = 1;  
}
```

And receive this message:



Our decoder will see that key 1 is repeated twice in the blob. The decoder is expected to use the last value. So here it will produce an instance of `Thing` with `a` set to 456.

Depending on the type of the duplicate field, the decoder will have a different behavior, as described in the encoding docs:

For numeric types and strings, if the same field appears multiple times, the parser accepts the last value it sees. For embedded message fields, the parser merges multiple instances of the same field, as if with the `Message::MergeFrom` method

Applications

Let's take a look at a few examples of needs that can be covered by taking advantage of protocol buffer encoding specifics.

Stream encoding

At Contentsquare, our mobile frameworks need to send payloads containing arrays of events to our servers. Each payload contains one encoded message as such:

```
message EventPayload {  
    repeated Event events = 1;  
}
```

One limitation we have is that the event payloads we send to our servers should be less than 1MB. If we have an `EventPayload` with `n` events, how do we know if adding one more event would get our payload over the 1MB limit? We would have to encode the whole payload every time we have an event just to know its current size. This is an inefficiency we can't afford.

There is a better way. Given what we now know about repeated field encoding, the following (pseudo code) is true:

```
encode(EventPayload(event_1, ..., event_n))  
== encode(EventPayload(event_1)) + ... + encode(EventPayload(event_n))
```

In other words, encoding an `EventPayload` of just one event, repeatedly for `n` events, and then concatenating the results, yields the same binary as encoding one `EventPayload` with the `n` events.

So, every time we produce an event, we can make an `EventPayload` of just that event, and check our ongoing payload blob to see if it would pass the 1MB limit. If not, it can be appended to the blob.

An added benefit we got from this is that at any time we only need to keep one byte blob in memory, and not an array of instances of `Event`, which helped us keep our memory footprint low.

More generally, we can incrementally encode any protocol buffer message property by property. We can then write each chunk to a stream, like a network stream, a file, or a compression stream.

- Protocol buffer decoders do not expect fields to be in any particular order. So we can stream encode fields of a message in any order.
- Some types of repeated protocol buffer fields, like numbers or boolean values, use a different encoding method called “packed encoding”. This is a more compact encoding where only one key is needed to encode all the values of a repeated field. In this case, encoding instances of the field one by one would yield a bigger binary than encoding them all at once. That being said, the resulting protocol buffer should still be correctly understood and parsed by any protocol buffer decoder.

Storing objects in files as protocol buffers

If we want to store objects in files, we need to serialize them. There are many benefits to doing this with protocol buffers, especially when we take advantage of what we learned on encoding.

Streaming

As seen before, we can encode and write to a file parts of an object. We can then encode more parts and append them to the file.

This means that we don’t need to wait until we know all the properties of an object before we write it. One benefit of that is that we can avoid using too much memory. Another benefit is that we can make sure any partial knowledge is immediately put on disk, saving it from a crash or sudden termination.

We can use optional fields to be able to read a file and know whether a field was set or not (for instance because there was a crash before it could be known).

Compatibility

Protocol buffer efforts in forward and backward compatibility can benefit us when using protocol buffers to write on files. There are many changes we can make to our schema while remaining able to read files we made with an old version of that schema: renaming fields and message types, adding fields... This can save us from needing to do “file migrations”.

In place field editing

As seen before, decoders should be able to accept fields appearing multiple times in an encoded message, and just use the last value or merge values. This means that it is possible to mutate fields of an object, in a file, without reading and rewriting the entire file. We can often just re encode a new value for the field, and append it to the file.

Encoding stability

How can we be sure of how stable protocol buffer encoding is going to remain? Is there really no chance that manually manipulating encoding, as we do above, could break? Could we end up producing unreadable data?

As stated before, maximum compatibility between data and programs using different schema versions is a key objective of protocol buffers. This is just as true for different versions of protocol buffer libraries. Even if the protocol buffer encoding specification came to change, data produced by old programs should remain readable by new programs.

It is possible that our manual encoding techniques could stop producing the same binary as newer encoders. But it should always remain readable by any decoder.

If there is a breaking change in protocol buffer encoding, it should result in an all new version of the protocol buffer language, such as `proto4` which would be declared in schemas.

Thanks to Jie Wang for helping explore all these ideas, and to Henrique Cesar for his review.

« SageDB, the Self-Assembling Database System (/2022/sagedb-the-self-assembling-database-system/) | How we scale out our ClickHouse cluster » (/2022/scaling-out-clickhouse-cluster/)

Want to join the fun? We are always hiring talented engineers (<https://contentsquare.com/careers/>)!